How program is organized
/schema.sql - contains the sql query to set up the tables in the database.

/src/ - all code files

- src/client: all code related to client application. *ClientApp.java* is the main entrypoint.
- src/server: all code related to server application. *ServerApp.java* is the main entrypoint. The ServerApp starts both the RMI service and the game service on separate threads.
- src/dto: contains classes used to make data transfer objects. This allows data to be passed around RMI interfaces in an organized fashion.

/lib/ - additional libraries + card images

How to compile code
Prerequisites

- Java 8
- Apache Ant
- Glassfish
- MySQL

Run the following exports:

```
export GLASSFISH_HOME=/path/to/glassfish5/glassfish
export MYSQL_CONNECTOR=/path/to/mysql-connector-j-8.2.0.jar
export CLASSPATH=$CLASSPATH:$GLASSFISH_HOME/lib/gf-client.jar:$MYSQL_CONNECTOR
```

Setup the database tables:

```
mysql -u root -p < schema.sql
```

Setup JMS resources:
Connection factory

Game Queue

Game Topic

Start rmiregistry

```
>> rmiregistry &
```

Start glassfish server and mysql server

```
>> ./GLASSFISH_HOME/bin/asadmin start-domain
```

```
>> sudo systemctl start mysql
```

To compile:

```
>> ant compile
```

After compilation, all .class files should be in /bin/

How to run code

```
>> cd bin
```

Run server application:

```
>> java -Djava.security.policy=../security.policy server.ServerApp
```

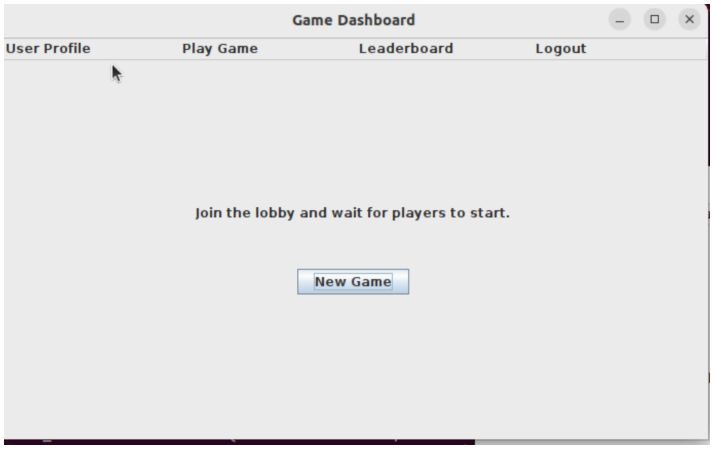
Run client application:
replace localhost with server ip (if server is running on different machine)

```
>> java client.ClientApp localhost
```

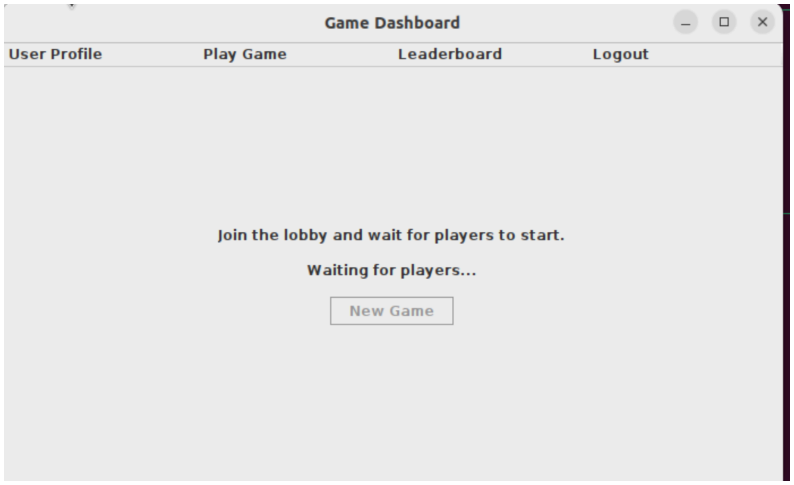
Screenshots of operations

Game-join phase

Play Game Screen



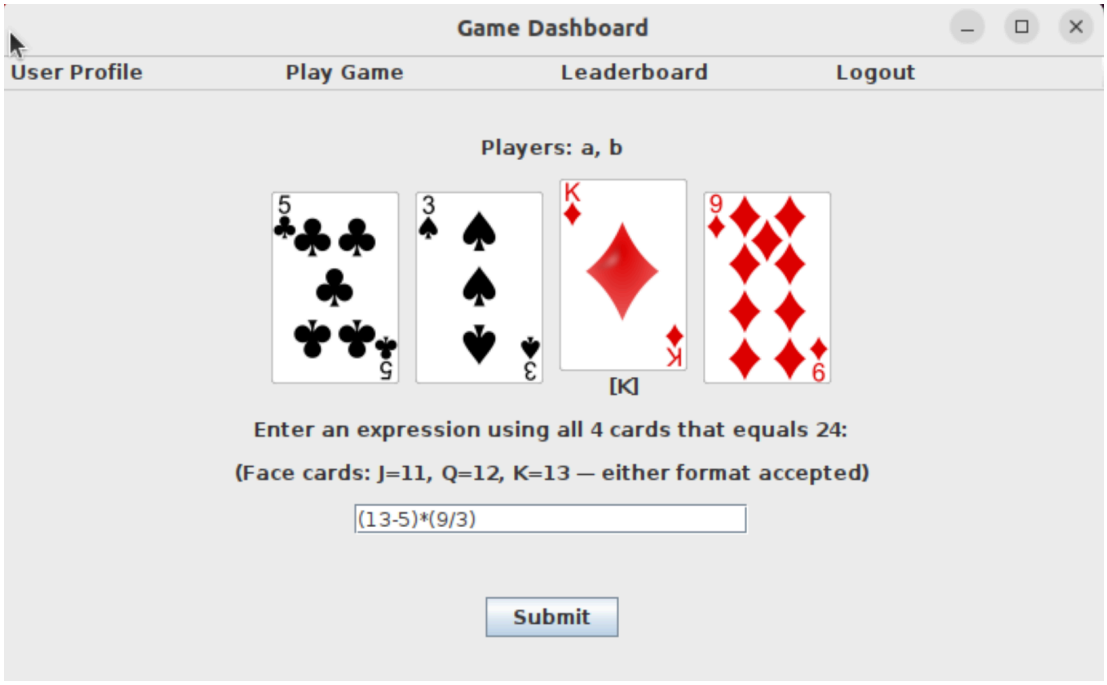
[[Press New Game Button]]
The client sends a message of format “JOIN: username” to the jms/JPoker24GameQueue. The game server listens to this queue and matches the player to a game. At the same time, the player subscribes to jms/JPoker24GameTopic so it can receive information from the server.



[[Player gets matched]]
The client sees a message that is of format “GAME_START:p1,p2,...|c1,c2,c3,c4” and sees their name as one of the players in the message. The client starts the game, displaying cards c1-c4.

Game playing phase

[[Player enters answer]]



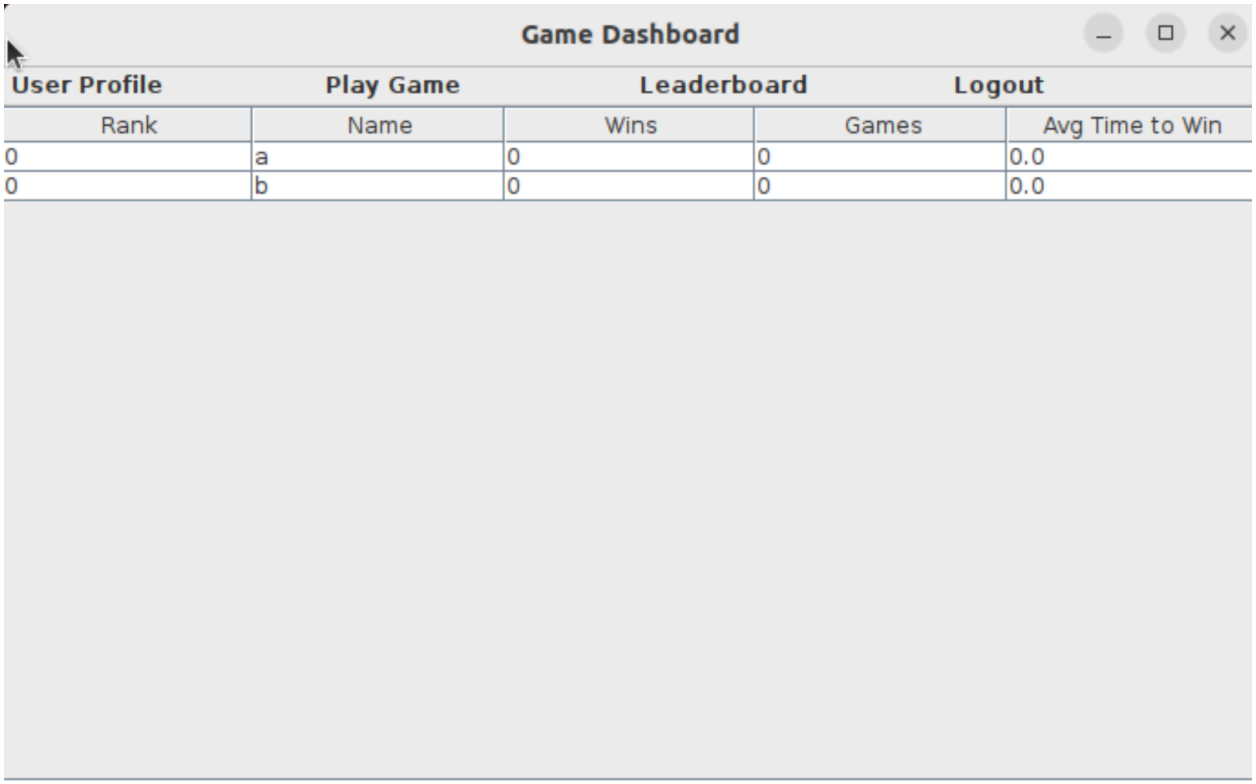
[[Player submits]]
The client submits the inputted answer as a message to the jms/JPoker24GameQueue, with format “ANSWER:username|expression”. The server checks whether the expression is valid and evaluates to 24 (if invalid, it is treated as a wrong answer). The server waits until all players have submitted their answers.

Game Over

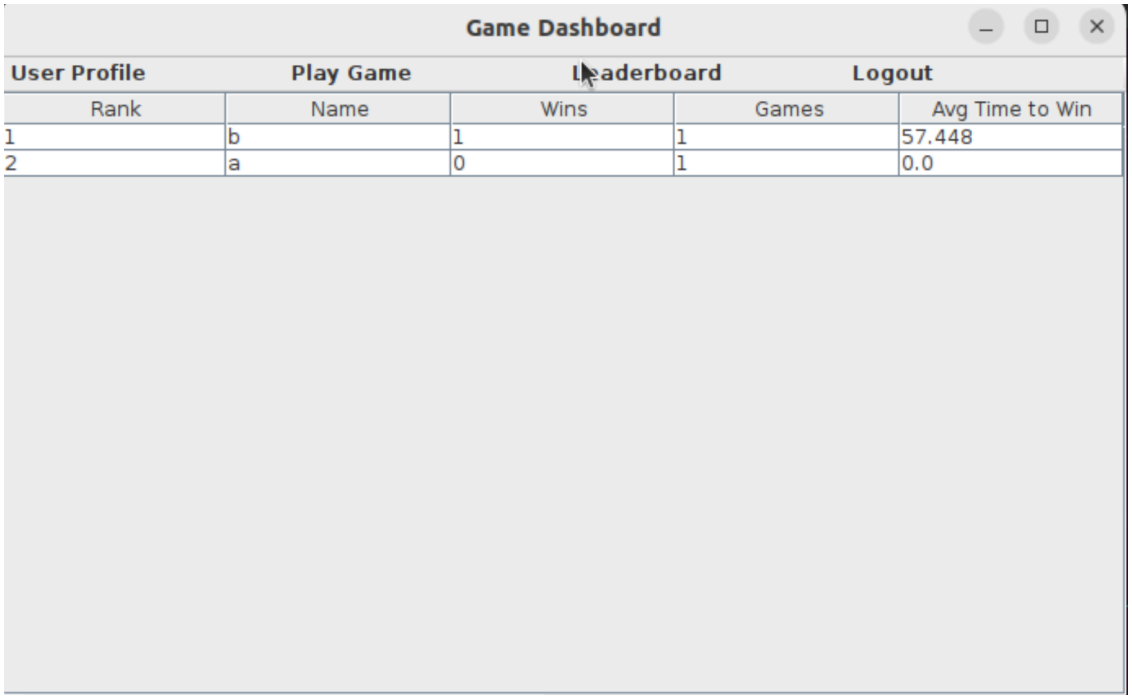
Once the server has received all the submissions, it sends a message of format “GAME WINNER: username” or “GAME_NO_WINNER” to the jms/JPoker24GameTopic. The client then displays the winner to the user.

Leaderboard update

Leaderboard **before game**



Leaderboard **after game** (player b won)



Once the game ends, the server updates the database via a transaction. This transaction includes:

1. Updating the games played field of each player in the session
2. Updating wins field of winner (if any)
3. Updating avg time to win filed of winner (if any)
4. Updating rank of every player according to number of wins.